

GHANA INSTITUTE OF MANAGEMENT AND PUBLIC ADMINISTRATION

School of Technology | Department of Computer Science

CS 305: Problem Set 1 (Automata & Regular Languages)

Weeks 1–3 | Total Marks: 100 | Weight: 7.5% of Final Grade

Instructions & Guidelines

- Submit your solutions as a cleanly typeset PDF or legible handwritten scan.
- Formal definitions require exact mathematical tuples (e.g., $(Q, \Sigma, \delta, q_0, F)$).
- State transition diagrams must clearly label start states (incoming arrow) and accept states (double circle).

Problem 1: DFA Construction & Formal Specifications (25 Marks)

Let the alphabet be $\Sigma = \{0, 1\}$. For each of the following languages, provide:

- (a) The formal 5-tuple $(Q, \Sigma, \delta, q_0, F)$ specifying all transition values.
- (b) The corresponding state transition diagram.
 - **Language L_1 :** The set of all strings where the number of 0's is a multiple of 3. (10 Marks)
 - **Language L_2 :** The set of all strings containing the substring 101. (15 Marks)

Problem 2: NFA Design & Powerset Construction (25 Marks)

Let $\Sigma = \{a, b\}$. Consider the language $L_3 = \{w \in \Sigma^* \mid \text{the 3rd symbol from the right end of } w \text{ is 'a'}\}$.

- (a) Design an intuitive 4-state NFA N that recognizes L_3 . (10 Marks)
- (b) Using the formal **Powerset (Subset) Construction Algorithm** (Sipser Section 1.2), convert NFA N into an equivalent DFA M . Show the step-by-step transition table for all reachable subset states. (15 Marks)

Problem 3: Regular Expressions & State Elimination (25 Marks)

Consider the DFA M_1 with states $Q = \{q_1, q_2\}$, alphabet $\Sigma = \{0, 1\}$, start state q_1 , accept state q_2 , and transitions: $\delta(q_1, 0) = q_1, \delta(q_1, 1) = q_2, \delta(q_2, 0) = q_1, \delta(q_2, 1) = q_2$.

- (a) Convert M_1 into a Generalized NFA (GNFA). (10 Marks)
- (b) Apply the GNFA state elimination procedure (Sipser Lemma 1.60) to eliminate internal states and derive the exact equivalent regular expression R . (15 Marks)

Problem 4: Modern Applied Case Study — Catastrophic ReDoS (25 Marks)

In July 2019, a major cloud security provider experienced a worldwide outage due to catastrophic backtracking in a Web Application Firewall (WAF) regex rule matching input strings against the pattern $R = (a^+)+b$.

- (a) Trace the number of computational backtracking steps an NFA-based backtracking regex engine performs on the non-matching input string $s = \underbrace{aaa \dots a}_{n \text{ times}}$. Prove that the running time is $\Omega(2^n)$.
(10 Marks)
- (b) Explain how converting R into an equivalent DFA via Thompson's algorithm or Powerset Construction solves the ReDoS vulnerability, and state the resulting worst-case time complexity on an input of length n . (15 Marks)